Health Transformation Review — Complete Testing, Usage & Deployment Guide

For operators, testers, and future users who are new to both the platform and the subject matter.

How to Use This Guide

This guide does two things at once. It walks you through every feature of the platform — from creating your first piece of content to deploying the application publicly — and it teaches you enough about healthcare transformation that you can evaluate whether the content and analysis is actually *good*, not just whether the buttons work.

Work through it in order. Each chapter builds on the last. Do not skip the Healthcare Primer (Chapter 1) even if you're in a hurry — it is what lets you catch errors in AI-generated content and know when the data is telling you something meaningful versus something wrong.

Estimated total time: 8–12 hours across multiple sessions.

You will need:

- The application running locally (`localhost:3000` frontend, `localhost:8000` backend)
- Access to Sanity Studio at `localhost:3000/studio`
- Access to the Supabase dashboard
- A Stripe account (free) — setup is covered in Chapter 2
- Your `.env.local` and `backend/.env` files with all credentials populated

Chapter 1 — Healthcare Transformation Primer

Read this before touching anything technical. It will make every subsequent chapter more useful.

1.1 What Is Healthcare Transformation and Why Does It Matter?

The American healthcare system spends more money per person than any other country on earth — roughly $14,000 per person per year as of 2025. Despite that spending, health outcomes in the United States are worse than most comparable wealthy nations on almost every major measure: life expectancy, maternal mortality, chronic disease burden, mental health access.

This is not primarily a medical problem. It is a *systems* problem. The way healthcare is paid for, organized, delivered, and measured creates structural incentives that produce poor outcomes at high cost.

Healthcare transformation refers to the deliberate process of changing those structures — the payment models, the delivery systems, the workforce practices, the technology infrastructure, and the equity frameworks — so that the system produces better outcomes, reaches more people, and operates sustainably.

This is not a political argument. It is an economic and clinical reality. A healthcare system that cannot keep nurses from burning out, cannot prevent patients from skipping care because they can't afford the deductible, and cannot coordinate care across providers is not a sustainable system regardless of political perspective.

The HTR platform exists to give healthcare leaders — hospital executives, state health officials, policymakers, and economists — the intelligence infrastructure to understand where the system is, where it is going, and what interventions are most likely to change the trajectory.

1.2 The Five Pillars — Explained Simply

The five pillars are the five structural forces that determine whether a healthcare system is transforming toward sustainability or drifting toward fragility. Think of them as five lenses through which to analyze any healthcare situation.

Pillar 1: Policy

What the rules allow and require.

Policy is the permission structure of healthcare. Before a hospital can bill for a telehealth visit, there has to be a law or regulation permitting it. Before a state can run a global budget model (where hospitals receive a fixed annual payment for a population rather than billing per procedure), there has to be statutory authority.

Policy determines what is possible. The most well-resourced, well-intentioned healthcare organization cannot transform its payment model if the regulatory environment won't allow it.

What this means for content quality: Policy articles on this platform should be grounded in specific legislation and rulemaking. "CMS issued a new rule" is not a policy article. "CMS's 2025 physician fee schedule rule reduces reimbursement for primary care evaluation and management codes by 3.4%, effective January 1, creating a $2.1 billion reduction in primary care revenue nationally" — that is a policy article.

Key terms to know:

- **CMS** (Centers for Medicare & Medicaid Services) — the federal agency that sets payment rules for Medicare and Medicaid, covering roughly 140 million Americans
- **APM** (Alternative Payment Model) — any payment model other than traditional fee-for-service. Includes capitation, global budgets, bundled payments, shared savings
- **Global budget** — a fixed annual payment to a hospital or health system to cover all care for a defined population. Vermont's AHEAD Model uses this approach
- **Scope of practice** — what clinical services a provider type (nurse practitioner, physician assistant, etc.) is legally permitted to perform. Expanding scope is a key workforce strategy

---

Pillar 2: Economics

The financial forces that determine whether transformation survives.

Healthcare transformation fails in the spreadsheet before it fails in the clinic. A care model that improves outcomes but loses money will be discontinued. An organization with negative operating margins cannot invest in transformation.

The economics pillar tracks the financial architecture of healthcare: how hospitals are paid, how much they spend on labor, how capital flows in and out of the sector, and whether the payment models that support better care are economically viable at scale.

What this means for content quality: Economic articles should include specific numbers. Operating margins. Per-capita spending. Wage inflation percentages. Revenue in risk-based contracts. Vague statements like "hospitals are under financial pressure" are not useful. "The median hospital operating margin fell from 3.1% in 2019 to -1.4% in 2022 before recovering to 0.8% in 2024, with rural hospitals averaging -2.3% versus academic medical centers at +4.1%" — that is useful.

Key terms to know:

- **Operating margin** — the percentage of revenue left after operating expenses. A 2% margin is thin. Negative margin means spending more than earning
- **Fee-for-service (FFS)** — the traditional payment model: each service delivered is billed separately. Creates incentives to do more, not better
- **Capitation** — a per-member-per-month payment to a provider to cover all care for a patient. Creates incentives to keep people healthy rather than treat them when sick
- **Value-based care (VBC)** — the broad category of payment models that tie reimbursement to outcomes rather than volume. The HTI dedicates an entire domain (15% weight) to VBC penetration
- **ACO** (Accountable Care Organization) — a group of providers that takes collective accountability for the cost and quality of care for a defined patient population

---

Pillar 3: Technology

The infrastructure layer that determines whether transformation can scale.

Technology is the infrastructure of healthcare transformation. Without interoperability — the ability for patient records to follow patients across providers — care coordination is nearly impossible. Without cybersecurity, digital transformation creates catastrophic risk. Without AI tools, quality improvement cannot scale.

What this means for content quality: Technology articles should name specific systems and standards. "Better EHRs" is not useful. "HL7 FHIR R4-compliant APIs enabling real-time patient data exchange between Epic and Cerner systems, as required by the 21st Century Cures Act's information blocking provisions" is useful.

Key terms to know:

- **EHR** (Electronic Health Record) — the digital system where clinical data lives. Epic and Cerner are the dominant vendors. Having an EHR is not the same as having interoperability
- **FHIR** (Fast Healthcare Interoperability Resources) — the technical standard that allows different healthcare systems to exchange data. Think of it as the common language
- **HIE** (Health Information Exchange) — an organization or network that enables health data sharing across providers and systems in a region
- **Prior authorization** — the process by which a payer must approve a treatment before a provider can deliver it. A major source of administrative burden and care delay
- **Telehealth** — care delivered remotely, including video visits, asynchronous messaging, and remote patient monitoring

---

Pillar 4: Clinical

The quality, access, and structure of actual care delivery.

This is where everything else has to show up. All the policy reform, economic innovation, and technology investment ultimately needs to manifest in better clinical care reaching more patients. The clinical pillar tracks readmission rates, preventable hospitalizations, access to specialty care, precision medicine adoption, and emerging care models like Hospital-at-Home.

What this means for content quality: Clinical articles should cite outcome data. Not "Hospital-at-Home programs improve patient satisfaction" but "Hospital-at-Home programs at five health systems in the 2023 CMS waiver cohort showed 30-day readmission rates of 6.8% versus 11.4% for matched inpatient cohorts, with patient satisfaction scores 14 points higher on a 100-point scale."

Key terms to know:

- **Readmission rate** — the percentage of patients who return to the hospital within 30 days of discharge. High readmission rates indicate poor discharge planning or inadequate follow-up care. CMS penalizes hospitals financially for high readmission rates
- **Preventable hospitalization** — hospitalization for conditions (asthma, diabetes, hypertension) that should have been manageable in primary care if the patient had access and compliance support
- **PQI** (Prevention Quality Indicator) — AHRQ's set of measures tracking preventable hospitalization rates by condition
- **HEDIS** — the standardized quality measurement set from NCQA, covering preventive care, chronic disease management, and behavioral health
- **Population health** — the practice of managing health outcomes across a defined group of people rather than treating individual patients episodically

---

Pillar 5: Equity

Whether transformation is reaching everyone.

Equity is not a social aspiration added onto the side of a clinical strategy. It is a measure of whether the transformation is real. A health system that improves outcomes for commercially insured, suburban patients while disparities grow in its rural and low-income populations is not transforming — it is stratifying.

Equity carries a 20% weight in the HTI — equal to Digital Maturity and Clinical Excellence — precisely because it is not optional. And it is consistently the lowest-scoring domain nationally. No state in the tracked dataset scores above 78 on the Social Determinants domain.

What this means for content quality: Equity articles should quantify disparities. Not "racial disparities in care persist" but "Black patients in this study were 1.7 times more likely to receive inadequate pain management following surgical procedures, a disparity that persisted after controlling for insurance status, comorbidities, and care setting."

Key terms to know:

- **SDOH** (Social Determinants of Health) — the non-clinical factors that affect health: housing, food security, transportation, income, social connection. These factors account for an estimated 30–55% of health outcomes
- **Health equity** — the state in which every person has a fair and just opportunity to achieve their highest level of health. Not equality (same for everyone) but equity (what each person needs)

- **Disparities** — measurable differences in health outcomes between population groups (racial, economic, geographic) that are not explained by clinical factors
- **CHW** (Community Health Worker) — a trained community member who bridges clinical care and community resources, often the most effective intervention for SDOH integration

---

1.3 The Health Transformation Index (HTI) — A Simple Explanation

The HTI is a score from 0 to 100 that measures how far a healthcare institution, region, or state has progressed in transforming its system. Think of it as a report card for healthcare transformation.

The score is built from six dimensions (called domains), each worth a certain percentage of the final score:

Domain	Weight	What It Measures in Plain English
Digital Maturity	20%	Can data flow where it needs to go? Is the system secure? Are AI tools in use?
Social Determinants	20%	Is the system closing outcome gaps across populations?
Clinical Excellence	20%	Are patients getting better care and fewer preventable hospitalizations?
Value-Based Care	15%	Is the payment model rewarding good outcomes rather than volume?
Patient Experience	15%	Do patients feel well-served and engaged?
Workforce Wellness	10%	Are clinicians burning out and leaving?

The four status levels:

Score	Status	What It Means
78–100	Leading	Ahead of the nation. A model for others
65–77	Improving	Above average. Positive momentum
50–64	Stable	Near national average. Progress exists but is uneven
Below 50	At Risk	Below average. Structural problems require urgent attention

The critical insight: A system can score well on some domains while struggling badly on others. California scores 90 on Digital Maturity but only 63 on Social Determinants. Vermont leads the nation on Value-Based Care (88) but trails Massachusetts on Patient Experience. The domain breakdown is more diagnostic than the composite score.

1.4 The HTR System Health Index (SHI) — A Simple Explanation

While the HTI measures individual institution/state transformation progress, the SHI measures the health of the *entire American healthcare economy*. It uses Q4 2019 (just before the pandemic) as a baseline of 100.

- SHI above 100 = the healthcare system is structurally healthier than before the pandemic
- SHI below 100 = the system is more fragile than the pre-pandemic baseline

It measures four forces:

- **Provider Stability (40%):** Are hospitals financially viable?
- **Payer Friction (30%):** Are claims being paid fairly and quickly?
- **Patient Access (20%):** Can patients afford and reach care?
- **Innovation Velocity (10%):** Is the system adopting transformative care models?

1.5 Vocabulary Reference Card

Keep this handy as you work through the rest of the guide. Every important term used in the platform is defined here.

Term	Simple Definition
ACO	Accountable Care Organization — a group of providers accountable for cost and quality of care for a population

Term	Simple Definition
AHEAD Model	Vermont's All-Payer Health and Economic Accountability under Development — a global budget model for Medicare/Medicaid/commercial payers
APM	Alternative Payment Model — any payment approach other than fee-for-service
Capitation	Fixed monthly payment per patient, regardless of services delivered
CHW	Community Health Worker
CMS	Centers for Medicare & Medicaid Services — federal payer and rule-setter
Deductible	Amount a patient pays out-of-pocket before insurance kicks in
DSO	Days Sales Outstanding — how long it takes a hospital to collect payment
EHR	Electronic Health Record
FHIR	Fast Healthcare Interoperability Resources — the data exchange standard
Global Budget	Fixed annual payment to cover all care for a population
HEDIS	Healthcare quality measurement set from NCQA
HIE	Health Information Exchange
Hospital-at-Home	Delivering acute-level hospital care in a patient's home
Operating Margin	% of revenue remaining after operating costs
PQI	Prevention Quality Indicators — measuring preventable hospitalizations
Prior Authorization	Payer approval required before delivering a service
PROM	Patient-Reported Outcome Measures — patients self-reporting health status
RHT	Rural Health Transformation — the $10B+ federal investment program
SDOH	Social Determinants of Health
Telehealth	Remote care delivery
Value-Based Care	Payment tied to outcomes, not volume
VBC	Value-Based Care (abbreviation)

Chapter 2 — Environment Setup and Verification

Before you can test anything, everything needs to be running. This chapter gets all services operational and verified.

2.1 What You're Running Locally

Your local setup has four active components:

Component	Location	What It Does
Next.js Frontend	`localhost:3000`	The website users see
FastAPI Backend	`localhost:8000`	The AI Analyst engine
Supabase	Cloud (your project)	User database, auth, vector storage
Sanity CMS	Cloud (`fxz10xl7`)	All editorial content

Stripe and email (Resend) will be tested in test mode — no real money or email delivery during testing.

2.2 Starting the Frontend

```
cd frontend
npm install          # Only needed first time
npm run dev
```

Expected output:

```
▲ Next.js 16.x.x
- Local: http://localhost:3000
✓ Ready in Xs
```

Open `http://localhost:3000` in your browser. You should see the HTR homepage with the header, news ticker, and sidebar.

If it fails:

- Missing `node_modules`: Run `npm install` first
- Port in use: Another process is on 3000. Run `lsof -i :3000` to find and kill it
- Environment variable error: Check that `frontend/.env.local` exists and has all required variables

---

2.3 Starting the Backend

```
cd backend
pip install -r requirements.txt    # Only needed first time
uvicorn main:app --reload --port 8000
```

Expected output:

```
🚀 HTR AI Brain v3 starting...
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000
```

Verify the backend is healthy:

```
curl http://localhost:8000/health
```

Expected response:

```
{
  "status": "ok",
  "index_ready": true,
  "model": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector",
  "auth_enabled": false
}
```

Interpret each field:

- `status: "ok"` — server is running
- `index_ready: true` — the AI knowledge base is built and ready. If `false`, wait 1–3 minutes for the index to build from scratch
- `vector_store: "pgvector"` — using Supabase for persistent storage. If `"local_json"`, the pgvector connection is failing (check `SUPABASE_DB_URL` in `backend/.env`)

- `auth_enabled: false` — in dev mode, any user can access the AI regardless of subscription. This is correct for local development

If `index_ready` stays `false` after 5 minutes: Check the backend terminal for error messages. Common causes:

- `OPENAI_API_KEY` missing or invalid — embeddings cannot be generated
- `SANITY_API_TOKEN` missing — Sanity content cannot be fetched
- `SUPABASE_DB_URL` incorrect — pgvector cannot be written to (will fall back to local, which may also fail)

2.4 Verifying Supabase Connection

Go to your Supabase dashboard. Confirm these tables exist under the Table Editor:

- `profiles`
- `user_roles`
- `subscriptions`
- `stripe_customers`
- `stripe_events`
- `rag_documents`

If any are missing, the database migrations have not been run. Contact the development setup documentation or run the seed scripts.

Verify the pgvector extension is enabled: In Supabase: Database → Extensions → search "vector" → confirm it shows as enabled.

2.5 Verifying Sanity Connection

Open `http://localhost:3000/studio` in your browser.

You should see the Sanity Studio interface with the left sidebar showing content types: Policy Analysis, Academy Module, Post, Case Study, etc.

If you see a blank page or authentication error:

- Verify `NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7` in `frontend/.env.local`
- Verify `SANITY_API_TOKEN` is a valid Sanity API token with editor access
- Check that the Sanity project exists at `sanity.io/manage`

2.6 Stripe Setup — From Zero

Stripe handles subscription payments. You need to configure it even for local testing because the application uses Stripe Checkout for plan upgrades.

Step 1: Create a Stripe Account

Go to `stripe.com` and create a free account. You do not need to provide business information for test mode.

After signing in, make sure you are in **Test Mode** — the toggle is in the top-left of the Stripe dashboard. In test mode, no real money moves. All card numbers are fake.

Step 2: Create Products and Prices

In Stripe: Product Catalog → + Add product

Create four products, each with a monthly and yearly price:

Product 1: Subscriber

- Product name: `Subscriber`
- Monthly price: `$29.00` USD, recurring monthly
- Yearly price: `$276.00` USD, recurring yearly (≈ $23/month)
- Note the price IDs (format: `price_...`) for each

Product 2: Student

- Product name: `Student`
- Monthly price: `$49.00` USD, recurring monthly
- Yearly price: `$468.00` USD, recurring yearly (≈ $39/month)

Product 3: Professional

- Product name: `Professional`
- Monthly price: `$99.00` USD, recurring monthly
- Yearly price: `$948.00` USD, recurring yearly (≈ $79/month)

You will end up with 6 price IDs total. Copy all of them.

Step 3: Add Price IDs to Environment Variables

In `frontend/.env.local`, set:

```
STRIPE_PRICE_SUBSCRIBER_MONTHLY=price_xxxx
STRIPE_PRICE_SUBSCRIBER_YEARLY=price_xxxx
STRIPE_PRICE_STUDENT_MONTHLY=price_xxxx
STRIPE_PRICE_STUDENT_YEARLY=price_xxxx
STRIPE_PRICE_PROFESSIONAL_MONTHLY=price_xxxx
STRIPE_PRICE_PROFESSIONAL_YEARLY=price_xxxx
```

Also set:

```
STRIPE_SECRET_KEY=sk_test_xxxx
```

Get the secret key from: Stripe dashboard → Developers → API keys → Secret key (starts with `sk_test_`).

Step 4: Set Up Stripe Webhooks for Local Testing

Stripe webhooks are how Stripe tells your application "a payment just completed." For local testing, you need the Stripe CLI to forward webhooks to your local server.

Install Stripe CLI:

```
# macOS
brew install stripe/stripe-cli/stripe

# Or download from stripe.com/docs/stripe-cli
```

Login:

```
stripe login
```

Start webhook forwarding:

```
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

This outputs a webhook signing secret (starts with `whsec_`). Copy it.

Add to `frontend/.env.local`:

```
STRIPE_WEBHOOK_SECRET=whsec_xxxx
```

Keep this terminal window open during all Stripe testing. It must be running for checkout to complete and roles to be granted.

Step 5: Verify Stripe Configuration

Restart the frontend after adding environment variables:

```
# Stop the running frontend (Ctrl+C) and restart
npm run dev
```

Go to `localhost:3000/pricing`. You should see three subscription plan cards (Subscriber, Student, Professional) with the correct prices and a monthly/yearly toggle.

If prices don't show or the page errors, the price IDs are incorrect.

2.7 Backend Environment Variables Checklist

Open `backend/.env` (or `backend/.env.example` for the template). Verify these are all set:

```
GROQ_API_KEY=           # From console.groq.com
OPENAI_API_KEY=         # From platform.openai.com (for embeddings)
SANITY_PROJECT_ID=fxz10xl7
SANITY_API_TOKEN=       # From sanity.io/manage → API → Tokens
SUPABASE_URL=           # https://[project-ref].supabase.co
SUPABASE_SERVICE_ROLE_KEY=  # From Supabase Settings → API
SUPABASE_DB_URL=        # PostgreSQL pooler URL (port 6543)
FRONTEND_URL=http://localhost:3000
```

Test each API key:

```
# Test Groq
curl -X POST https://api.groq.com/openai/v1/chat/completions \
  -H "Authorization: Bearer $GROQ_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"llama-3.3-70b-versatile","messages":
[{"role":"user","content":"hello"}],"max_tokens":10}'
# Should return a JSON response with content

# Test OpenAI embeddings
curl https://api.openai.com/v1/embeddings \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"text-embedding-3-small","input":"test"}'
# Should return a JSON response with an "embedding" array
```

2.8 Full System Status Checklist

Before proceeding, confirm all of the following:

- [ ] Frontend running at `localhost:3000`
- [ ] Backend running at `localhost:8000`
- [ ] `curl localhost:8000/health` returns `"status": "ok"` and `"index_ready": true`
- [ ] Sanity Studio loads at `localhost:3000/studio`
- [ ] Supabase tables exist and pgvector is enabled
- [ ] Stripe products and prices created, price IDs in env vars
- [ ] `stripe listen` running in a separate terminal
- [ ] Frontend restarted after env var changes

Chapter 3 — Creating Content with AI

This chapter teaches you how to generate high-quality healthcare transformation content using AI prompts, how to evaluate the output, and how to publish it to Sanity.

3.1 Understanding the Content Types

The platform hosts eight main types of editorial content, each serving a different purpose:

Content Type	Purpose	Who Reads It
Policy Analysis	In-depth analysis of legislation, regulation, or policy developments	Senior leaders, policymakers
Post (Blog)	Shorter editorial commentary and news analysis	General subscribers
Academy Module	Structured educational content organized into courses	Students and professionals seeking certification
Case Study	Real-world examples of transformation in practice	Healthcare leaders seeking applicable lessons
Definition	Glossary entries for key terminology	Anyone new to the subject
Analyst Note	Short (2–3 sentence) current-event signals from named analysts	All subscribers
Webinar	Information about live or recorded events	Education-focused subscribers
Report	Longer-form research documents	Researchers and senior analysts

---

3.2 What Makes Healthcare Content High Quality

Before generating any content, understand what you're aiming for. Use these standards to evaluate every piece of AI-generated content before publishing.

Standard 1: Specificity over generality

- Bad: "Value-based care models are improving outcomes across the country."
- Good: "Accountable Care Organizations participating in the Medicare Shared Savings Program generated $1.8 billion in net savings in 2023, with 67% of participating ACOs earning shared savings payments."

Standard 2: Named sources and primary data

- Bad: "Research shows that hospital-at-home programs work."
- Good: "A 2023 JAMA study of Brigham and Women's Hospital's Acute Care for Elders at Home program showed 30-day readmission rates of 8.6% compared to 15.4% for matched inpatient controls ($p<0.001$)."

Standard 3: Explicit pillar connections Every piece of content should clearly connect to one or more of the five pillars. A policy analysis about CMS rulemaking affects Policy (obviously) but may also affect Economics (payment rates) and Technology (prior authorization reform). Good content makes these connections explicit.

Standard 4: Actionable for the audience The audience is healthcare executives and policymakers. Content should tell them what to *do* or *think about differently*, not just what is happening. The question to ask: "So what?" A CFO reading this should know whether this changes her organization's financial planning.

Standard 5: No unverifiable claims AI models can confidently generate plausible-sounding but fabricated statistics. Any specific number — a dollar figure, a percentage, a date — should be verifiable against a named source. If you cannot verify it, flag it for human review before publishing.

3.3 AI Content Prompt Templates

Template A: Policy Analysis Article

Use this when creating a deep-dive analysis of legislation, regulation, or policy development.

```
You are a senior health policy analyst writing for the Health Transformation Review,
a publication for healthcare executives, CFOs, and policymakers.

Write a policy analysis article about: [TOPIC]

Requirements:
- Length: 600–900 words
- Cite specific legislation, rule numbers, or regulatory documents by name
- Include at least three specific data points (dollar amounts, percentages, dates)
- Explain what this policy change means for: (1) hospital operators, (2) payers, (3)
patients
- Connect explicitly to at least two of the five pillars: Policy, Economics,
Technology, Clinical, Equity
- End with a "What to Watch" section: one paragraph on the 2–3 leading indicators
that will reveal whether this policy achieves its stated goals
- Write for an audience of experienced healthcare leaders who do not need basic
terms defined
- Tone: analytical, direct, no hype

Pillar: [Policy / Economics / Technology / Clinical / Equity]
Subcategory: [e.g., Regulation & Legislation / Value-Based Care Models / AI &
Machine Learning / etc.]
Impact Level: [Critical / High / Medium]
```

Example topics to test with:

- "The CMS 2025 Hospital Outpatient Prospective Payment System final rule and its impact on rural hospital margins"
- "Vermont Act 167 and the AHEAD Model: five years of global budget implementation"
- "The No Surprises Act: two years of implementation data and what it reveals about payer-provider friction"

Template B: Academy Module — Foundational Level

```
You are writing educational content for the HTR Academy, a structured learning
platform
for healthcare professionals entering the field of health transformation.

Write a Foundational-level academy module on: [TOPIC]

Your audience: Healthcare professionals (nurses, administrators, mid-level managers)
who understand how care is delivered but have no background in health economics,
policy analysis, or systems reform.

Structure:
1. Opening (2–3 paragraphs): What this module covers, why it matters, what you will
learn
2. Core Concept 1 — [Name it]: Define the concept in plain language. No jargon
without definition.
3. Core Concept 2 — [Name it]: Same approach.
4. Core Concept 3 — [Name it]: Same approach.
5. A Real Example: One concrete, named real-world illustration of these concepts in
action
6. Key Takeaways: 3–5 bullet points
7. Next Steps: One paragraph bridging to the next topic in the learning sequence

Learning Objectives (write 3–4 starting with action verbs):
- "Describe..."
- "Explain the relationship between..."
- "Identify..."

Pillar: [one of the five pillars]
Level: Foundational
Estimated Read Time: 8–12 minutes
```

Template C: Case Study

```
You are writing a case study for the HTR Academy Case Study Library.
Case studies document real organizations navigating healthcare transformation
challenges.

Write a case study about: [ORGANIZATION OR PROGRAM]

Structure:
1. Context (1 paragraph): The organization, its size/type, and the challenge it
faced
2. The Problem (2–3 paragraphs): What structural barrier or opportunity was at
stake.
   What would have happened if nothing changed.
3. The Approach (2–3 paragraphs): What they did. Specific initiatives, partnerships,
   investments, or policy changes. Named programs, dollar amounts, timelines.
4. The Outcome (1–2 paragraphs): What happened. Specific measurable results with
   timeframes. Be precise — percentages, dollar figures, patient counts.
5. Lessons Learned (3–5 bullets): What other organizations can take from this.
   What worked, what was harder than expected, what preconditions were necessary.

Pillar connection: Explicitly name which of the five pillars this case study
illustrates
and how they interact.

Do not fabricate data. If you are uncertain of specific numbers, write "[VERIFY:
estimated X]"
so the editor knows to check.
```

Template D: Glossary Definition

```
Write a glossary definition for the HTR Academy Glossary.

Term: [TERM OR ACRONYM]

Requirements:
- First sentence: Define the term in plain language and expand any acronym
- Second sentence: Explain why it matters in healthcare transformation
- Optional third sentence: Give one concrete example or contrast with a related term
- Maximum 120 words total
- Avoid using the term being defined in the definition itself
- Tag with relevant pillars: [list which of the five pillars this term belongs to]
```

Template E: Analyst Note (The Signal)

```
Write an analyst note for the HTR Signal column.

Topic: [CURRENT EVENT OR DATA POINT]

Requirements:
- Maximum 3 sentences total
- Lead with the finding, not the background
- Written for a senior healthcare executive audience — no basic definitions
- Bold 1–2 key phrases
- Headline: maximum 8 words
- Tone: urgent, analytical, direct

Format:
Headline: [Your headline]
Note: [Your 3-sentence note]
Author: [Analyst Name — use a realistic healthcare analyst name]
```

3.4 Evaluating AI-Generated Content Before Publishing

Run every AI-generated piece through this checklist:

Accuracy checks:

- ☐ Are all dollar figures, percentages, and dates plausible and source-citable?
- ☐ Are program names (AHEAD Model, MSSP, HEDIS, etc.) spelled and used correctly?
- ☐ Does the policy reference actually exist and say what the article claims?
- ☐ Are the pillar connections logical — does this content actually belong to the assigned pillar?

Quality checks:

- ☐ Would a hospital CFO reading this learn something actionable?
- ☐ Is there at least one specific, named real-world example?
- ☐ Is the writing direct? (Cut anything that could be removed without losing meaning)
- ☐ Does the conclusion say more than "more work needs to be done"?

Red flags — reject or heavily edit if:

- The article makes a strong claim with a round number ("studies show 50% improvement") with no source
- The writing is vague ("healthcare stakeholders are increasingly aware of...")
- The article explains a concept but never says what it means for the reader
- Proper nouns are invented (CMS program names that don't exist, bills with wrong numbers)

3.5 Publishing to Sanity Studio — Step-by-Step

Open `http://localhost:3000/studio`

Creating a Policy Analysis

1. In the left sidebar, click **Policy Analysis**
2. Click **+ New Policy Analysis** (top right)
3. Fill in the required fields:

   Title — The article headline. Keep it under 80 characters. Lead with the policy name, not a vague topic.

   - Good: "CMS 2025 Physician Fee Schedule: The $2.1B Primary Care Impact"
   - Bad: "New CMS Rules Could Change Healthcare"

   Slug — Auto-generates from the title. Click "Generate" if it doesn't. Clean it up if the auto-version is ugly. The slug becomes the URL: `/policy/[slug]`

   Pillar — Select from: Policy, Economics, Technology, Clinical, Equity. Use the dominant pillar even if multiple apply.

   Subcategory — Select the most specific fit from the dropdown. This determines how the article is sorted and filtered.

   Status — Active (published policy), Proposed (still in legislative process), or In Committee

   Impact Level — Critical (affects billions in spending or millions of patients), High (significant but contained), Medium (important but narrow)

   Summary — 2–3 sentences. This appears on the listing page and is what users read before clicking through. It is also indexed first by the AI Analyst. Make it specific — include the most important number or finding.

   Body — The full article content. Paste from your AI-generated text. Use the formatting toolbar for headings (H2 for main sections), bold, and lists.

4. Click **Publish** (not just Save)

Important: A document must be Published (not just saved as a draft) to appear on the site. The green "Published" indicator in the top-right confirms this.

Creating an Academy Module

1. Click **Academy Module** in the sidebar → **+ New**
2. Required fields:

   Course Title — Exact string matching the parent course. If this module belongs to "Value-Based Care Fundamentals," every module in that course must have `courseTitle = "Value-Based Care Fundamentals"` (case-sensitive, character-exact)

   Module Number — The position in the course (1, 2, 3...)

   Total Modules — How many modules in the full course (e.g., 5)

   Pillar and Level — Match the course's pillar and level

   Estimated Read Time — In minutes (e.g., 10)

   Learning Objectives — List format. Each starts with an action verb:

   - "Analyze the financial structure of an Accountable Care Organization"
   - "Explain the difference between capitation and shared savings models" These are indexed by the AI Analyst separately from the body text.

   Summary — 3–4 sentences for the listing page

   Body — Full module content

   Prev/Next Module Slugs — Leave blank on first creation. After creating all modules in a course, go back and wire these up (see the Course Wiring section below)

3. Publish each module
4. Note the auto-generated slug after publishing

Wiring Module Navigation

After all modules in a course are published:

1. Open Module 1 → set `nextModuleSlug` to the slug of Module 2 → Publish
2. Open Module 2 → set `prevModuleSlug` to Module 1's slug, `nextModuleSlug` to Module 3's slug → Publish
3. Continue through the sequence
4. Last module: set `prevModuleSlug` only (no `nextModuleSlug`)

Verify navigation: go to `localhost:3000/academy/modules/[first-module-slug]` and confirm Next/Previous buttons work.

Publishing an Analyst Note (The Signal)

Analyst Notes appear in the left sidebar across all pages. Maximum two active at a time.

1. Click **Analyst Note** → **+ New**
2. Fill in:
   - **Headline** — Maximum 50 characters
   - **Body** — 2–3 sentences, Portable Text (bold and italic supported)
   - **Author** — Analyst name
   - **Is Active?** — Check this box to make it visible. Only two can be active simultaneously. Uncheck old notes rather than deleting them.
3. Publish

3.6 Verifying Content on the Live Site

After publishing each piece of content:

Content Type	Where to Verify
Policy Analysis	`localhost:3000/policy/[slug]`
Academy Module	`localhost:3000/academy/modules/[slug]`

Content Type	Where to Verify
Case Study	`localhost:3000/academy/case-studies/[slug]`
Glossary Definition	`localhost:3000/academy/glossary` (search for the term)
Analyst Note	Home page or any content page — check the left sidebar
Webinar	`localhost:3000/academy/webinars`

If content doesn't appear:

1. Confirm it is Published in Studio (not Draft)
2. Confirm the slug field is set
3. Hard refresh the browser (Cmd+Shift+R)
4. Check the browser DevTools Network tab for failed API calls to `api.sanity.io`

Chapter 4 — User Accounts, Roles, and Subscription Testing

This chapter covers creating users, the full Stripe checkout flow, role management, and testing what each subscription tier can access.

4.1 The Role System — Understanding Access Tiers

The platform has six roles in ascending order of access:

Role	Who Has It	What They Can Access
`free`	Any registered user with no subscription	Public content, limited navigation
`subscriber`	$29/month or $276/year	Full article library, AI Analyst, all five pillars
`student`	$49/month or $468/year	Everything in Subscriber + academy modules and certifications
`professional`	$99/month or $948/year	Everything in Student + HTI Dashboard, deep analytics
`advisory`	Custom/contact	Everything in Professional + deeper AI Analyst mode, custom research
`admin`	Internal team only	Everything

The AI Analyst requires `subscriber` or higher. This is the primary paywall. Free users cannot access the chat interface.

4.2 Creating a Test User

Go to `localhost:3000` → Click "Login" in the header → Click "Sign Up"

Create a test account:

- Email: `test@example.com` (or use a real email you control for email verification)
- Password: Something you'll remember

After signup, Supabase creates a user in `auth.users`. A `profiles` row is also created automatically.

Verify in Supabase: Table Editor → `auth.users` (under Authentication) → find your test email Table Editor → `profiles` → verify a row exists with your user UUID

At this point the user has no role — they are treated as `free`.

4.3 Testing the Subscription Flow (Stripe Test Mode)

Ensure the Stripe CLI `stripe listen` terminal is still running.

1. Log in as your test user
2. Go to `localhost:3000/pricing`
3. Click "Subscribe" on the Subscriber plan (Monthly)
4. You are redirected to Stripe Checkout

Use this Stripe test card: `4242 4242 4242 4242`

- Expiry: Any future date (e.g., 12/28)
- CVC: Any 3 digits (e.g., 123)
- Name: Any name
- Postal code: Any 5 digits (e.g., 12345)

5. Click "Subscribe"
6. Watch the `stripe listen` terminal — you should see:

```
--> checkout.session.completed [evt_...]
<-- [200] POST http://localhost:3000/api/stripe/webhook
```

The 200 confirms the webhook was received and processed.

7. Check the Supabase `user_roles` table — you should now see a row: `{ user_id: "...", role: "subscriber" }`
8. Check the `subscriptions` table — you should see a row with `status: "active"` and `plan: "subscriber"`

If the webhook shows an error (non-200 response):

- Check the frontend terminal for error logs on `/api/stripe/webhook`
- Verify `STRIPE_WEBHOOK_SECRET` in `.env.local` matches the `whsec_` value from the `stripe listen` output
- Verify `STRIPE_SECRET_KEY` is set and starts with `sk_test_`

4.4 Verifying Role-Gated Access

Test what each role can and cannot access:

Test 1: Free user cannot access AI chat

1. Log out (or use a different browser/incognito with a different test account with no role)
2. Go to `localhost:3000/chat`
3. Expected: Redirect to pricing page or access denied message
4. Pass condition: The AI chat interface is not usable

Test 2: Subscriber can access AI chat

1. Log in as the user who just completed the Stripe checkout
2. Go to `localhost:3000/chat`
3. Type a test message: "What is the AHEAD Model in Vermont?"
4. Expected: A response from the AI Analyst
5. Pass condition: Response arrives and references Vermont healthcare content

Test 3: Manually grant a role To test higher tiers without paying, grant a role directly in Supabase:

```
-- In Supabase SQL Editor
INSERT INTO user_roles (user_id, role)
VALUES ('paste-your-user-uuid-here', 'professional')
ON CONFLICT (user_id, role) DO NOTHING;
```

Get the UUID from `auth.users` in Supabase. After running this, log out and back in, then try accessing the HTI Dashboard.

4.5 Testing Subscription Cancellation

In Stripe test dashboard → Customers → find your test customer → Subscriptions → Cancel

Watch the `stripe listen` terminal for `customer.subscription.deleted`

Check Supabase `user_roles` — the `subscriber` role should be removed and replaced with `free`

4.6 Testing Failed Payment

In Stripe: use card number `4000 0000 0000 0341` (charge always fails after successful authorization)

This triggers an `invoice.payment_failed` webhook. The user's `subscriptions.status` should update to `past_due` in Supabase.

Chapter 5 — RAG Indexing and AI Chat Testing

This is where you test the intelligence core of the platform. The AI Analyst is only as good as its knowledge base. This chapter covers building the index, testing systematically, and evaluating response quality.

5.1 What the AI Analyst Is — and Is Not

The AI Analyst is not a general-purpose chatbot. It is a Retrieval-Augmented Generation (RAG) system — which means it answers questions by:

1. Finding the most relevant documents in the HTR knowledge base
2. Sending those documents as context to a language model
3. The language model generates a response *based on that specific context*

This means the AI's quality is directly tied to the quality and comprehensiveness of the content in the knowledge base. If you ask about a topic that has no indexed content, the AI will either say it doesn't know or (worse) improvise an answer that isn't grounded in HTR material.

The AI's job is to cite, synthesize, and explain — not to invent.

A high-quality AI Analyst response:

- Names specific documents ("According to the Value-Based Care Fundamentals module...")
- Cites specific data points from indexed content
- Distinguishes between what is established (from indexed sources) and what is its interpretation
- Acknowledges when it doesn't have enough indexed information on a topic

A low-quality AI Analyst response:

- Makes claims without citing any source
- Gives generic answers that could come from any LLM
- Fabricates specific statistics that aren't in the indexed content
- Fails to reference HTR-specific content when it exists

5.2 Building the Knowledge Base

Before testing the AI, rebuild the knowledge base to include all content you've published in Sanity plus the PDF documents in `backend/data/`.

What gets indexed:

- All published Policy Analysis documents from Sanity
- All published Academy Modules from Sanity
- All published Case Studies, Definitions, Analyst Notes, Webinars, Reports
- All PDF files in `backend/data/` (currently: Vermont Act 167 presentation, Health Economics reference, Wyman Report)

Trigger the re-index:

```
curl -X POST http://localhost:8000/api/ingest
```

Expected response:

```
{"message": "Index build started in background"}
```

Watch the backend terminal for progress:

```
🔗 Fetching Sanity CMS content...
📄 Loading 3 PDF(s) from data/...
⏳ Embedding N documents...
✅ Index built and stored in Supabase pgvector
```

This takes 1–3 minutes depending on content volume.

Verify the index built:

```
curl http://localhost:8000/health
```

Confirm `"index_ready": true`

Verify document count in Supabase:

```
-- In Supabase SQL Editor
SELECT COUNT(*), metadata->>'source' as source
FROM rag_documents
GROUP BY source
ORDER BY count DESC;
```

You should see rows for each content type that had published documents.

5.3 AI Chat Testing — The Question Battery

Test the AI Analyst with these questions in sequence. For each, evaluate not just whether you get an answer, but whether the answer is grounded in indexed content and clinically/policy-accurate.

Series 1: Vermont and AHEAD Model (high-indexed content)

Ask each of these and evaluate:

1. "What is Vermont's AHEAD Model and how does it differ from traditional Medicare payment?"

   Expected quality indicators:

   - References Vermont Act 167
   - Explains global budget concept specifically (not just "value-based care")
   - Mentions the all-payer structure (Medicare, Medicaid, commercial)
   - May reference the Act 167 PDF or Vermont policy analysis articles if indexed

2. "What is Vermont's HTI score and what are its strongest and weakest domains?"

   Expected quality indicators:

   - References the Vermont HTI data (composite 82, Leading status)
   - Identifies VBC (88) as a strength
   - Should identify Patient Experience (82) or Workforce (76) as relatively weaker domains
   - Should note the national average for comparison (69)

3. "Why does Vermont score so high on Value-Based Care compared to states like Texas?"

   Expected quality indicators:

   - Contrasts Vermont (88) with Texas (52) on VBC
   - Should connect this to policy environment — Vermont's global budget authority vs. Texas not having Medicaid expansion
   - Should mention Act 167 as the enabling legislation

Series 2: Healthcare Economics (foundational knowledge)

4. "Explain the difference between fee-for-service and capitation payment models."

   Expected quality indicators:

   - Clear, accurate definitions of both
   - Should explain the incentive difference (volume vs. outcomes)
   - Ideally references specific HTR academy content on VBC if indexed

5. "What does a hospital operating margin of -2% mean and why does it matter for transformation?"

   Expected quality indicators:

   - Explains that negative margin = spending more than earning
   - Connects to transformation: organizations with negative margins cannot invest in transformation initiatives
   - Should reference the economic preconditions for sustainable transformation

Series 3: Equity and SDOH (important for evaluating the equity pillar content)

6. "What are social determinants of health and why do they affect clinical outcomes?"

   Expected quality indicators:

   - Lists specific SDOH factors (housing, food, transportation, income, social connection)
   - Quantifies their impact (they account for 30–55% of health outcomes)
   - Connects SDOH to clinical metrics (preventable hospitalizations, readmission rates)

7. "Texas has a low HTI equity score. What specific metrics drive this and what interventions could improve it?"

   Expected quality indicators:

   - References Texas's Social Determinants score (44)
   - Should note high uninsured rate (18.4%) and Medicaid non-expansion
   - Should discuss the high maternal mortality rate (34.2 per 100k)
   - Should suggest specific interventions (CHW expansion, SDOH screening)

Series 4: AI Knowledge Boundary Test

8. "What is the current federal funds rate and how does it affect hospital financing?"

   Expected quality indicators:

   - This is NOT in the HTR knowledge base
   - The AI should acknowledge it doesn't have current Federal Reserve data
   - It may offer a general framework for how interest rates affect healthcare capital costs
   - **Red flag:** If the AI fabricates a specific current rate, that is a quality failure

9. "Who won the 2024 US presidential election?"

   Expected quality indicators:

   - This is outside the AI Analyst's scope
   - Should decline to answer or redirect to healthcare-relevant policy implications
   - Should not fabricate an answer

Series 5: Multi-Turn Conversation

Ask these in sequence (don't refresh between them):

10. "Tell me about hospital-at-home programs."
11. "What are the typical clinical outcomes compared to inpatient care?"
12. "What policy changes were necessary to make these programs financially viable?"
13. "Which states in our dataset have the best clinical metrics to support hospital-at-home expansion?"

Expected quality indicators:

- The AI maintains context across turns (knows question 12 is still about hospital-at-home)
- The answers build coherently — it doesn't reset to generic definitions after turn 1
- Turn 4 connects the clinical metrics data (preventable hospitalization, primary care density) to HaH expansion readiness

---

5.4 Evaluating AI Response Quality — Scoring Rubric

For each question answered, score 1–3 on each dimension:

Dimension	1 — Poor	2 — Acceptable	3 — Excellent
Accuracy	Contains verifiable errors	Technically correct but imprecise	Accurate with appropriate caveats
Grounding	No document citations	Some vague reference to HTR content	Explicitly names documents and quotes data
Depth	Surface-level generic answer	Covers the main points	Addresses nuance, tradeoffs, and implications
Relevance	Goes off-topic	Mostly on-topic	Precisely addresses the question asked
Actionability	No actionable insight	Some practical information	Clear "so what" for a healthcare executive

A score of 12–15 is excellent. Below 8 warrants investigation and content improvement.

5.5 Follow-Up Suggestions Testing

After each AI response, the platform generates three follow-up question suggestions. Verify:

1. The suggestions appear within 2–3 seconds of the response completing
2. The suggestions are topically relevant (not generic)
3. Clicking a suggestion pre-fills it into the chat input
4. The AI responds coherently to the clicked suggestion

5.6 Chat Interface Feature Testing

Test each UI feature:

Feature	How to Test	Pass Condition
Streaming response	Ask any question	Text appears word-by-word, not all at once
Stop generation	Click Stop while response is streaming	Response stops immediately
Regenerate	Click regenerate on a completed response	Gets a new response to the same question
Copy response	Click copy icon on a response	Response text is in clipboard
Download transcript	Click download button	A text file with conversation history downloads
Thumbs up/down	Rate a response	Visual confirmation registers
Conversation persistence	Refresh the page	Previous conversation is restored from localStorage
Clear conversation	Click clear/new chat	Conversation resets, localStorage cleared

Chapter 6 — Academy: Courses, Modules, and Learning Tracks

This chapter covers building complete learning content and understanding what educational quality looks like in healthcare transformation.

6.1 The Academy Structure — How It Connects

```
Learning Track (e.g., "Policy Analyst Track")
  └── Course 1: "Health Policy Fundamentals"
        └── Module 1: "What Is Healthcare Policy?"
        └── Module 2: "How CMS Makes Rules"
        └── Module 3: "Reading a Federal Register Notice"
  └── Course 2: "Payment Reform"
        └── Module 1: "From Fee-for-Service to Value"
        ...
```

A learning track is a curated path through multiple courses. A course is a structured sequence of modules. A module is a single lesson.

6.2 What "Foundational," "Intermediate," and "Advanced" Mean

Foundational — The reader is intelligent but has no prior knowledge of this domain. Define every term on first use. Use analogies. Answer "what is this and why does it exist?" before explaining how it works.

Intermediate — The reader knows the vocabulary and basic concepts. Focus on mechanisms, tradeoffs, and implementation considerations. Answer "how does this actually work and what can go wrong?"

Advanced — The reader is a practitioner who needs strategic and analytical tools. Focus on edge cases, multi-stakeholder dynamics, quantitative frameworks, and decision-making under uncertainty. Answer "how do I use this to make better decisions?"

Test this yourself: Take the first module of any course you create and ask: "Could a smart hospital floor nurse with no MBA understand this in a single sitting?" If yes, it's Foundational-appropriate. If it assumes knowledge of APM financing structures, it's Intermediate or above.

6.3 Building Your First Complete Course

Let's build a 3-module Foundational course on Value-Based Care as a practical test.

Course: "Value-Based Care: From Fee-for-Service to Outcomes-Driven Payment"

Step 1: Create the Course Document

In Sanity Studio → Courses → + New Course

```
Title: Value-Based Care: From Fee-for-Service to Outcomes-Driven Payment
Description: A three-module introduction to alternative payment models and why
they represent the most significant structural shift in American healthcare since
the creation of Medicare.
Pillar: Economics
Level: Foundational
```

Publish it.

Step 2: Generate Module 1 Content with AI

Use Template B from Chapter 3 with:

```
Topic: Why fee-for-service payment creates misaligned incentives in healthcare
Pillar: Economics
Level: Foundational
Course: Value-Based Care: From Fee-for-Service to Outcomes-Driven Payment
Module number: 1 of 3
```

After reviewing and editing the AI output, create the module in Sanity with:

- `courseTitle`: "Value-Based Care: From Fee-for-Service to Outcomes-Driven Payment"
- `moduleNumber`: 1
- `totalModules`: 3
- `level`: Foundational
- `pillar`: Economics

Step 3: Generate and Create Modules 2 and 3

Module 2 topic: "What are Alternative Payment Models? Capitation, bundled payments, and shared savings explained"

Module 3 topic: "Accountable Care Organizations: How they work, what they've achieved, and where they're headed"

Step 4: Wire the Navigation

After all three modules are published, note their slugs and wire them:

- Module 1: nextModuleSlug → Module 2's slug
- Module 2: prevModuleSlug → Module 1, nextModuleSlug → Module 3
- Module 3: prevModuleSlug → Module 2

Step 5: Verify the Course

Navigate to `localhost:3000/academy/modules/[module-1-slug]`

Check:

- "Module 1 of 3" displays correctly
- Next button goes to Module 2
- Back on Module 2, Previous goes to Module 1 and Next goes to Module 3
- Module 3 shows Previous but no Next button

6.4 Glossary Management

The glossary at `localhost:3000/academy/glossary` is both a user reference and an AI Analyst source.

Building out the glossary: Using Template D from Chapter 3, create definitions for the following core terms (minimum viable glossary for the platform to feel complete):

Essential definitions to create:

- Alternative Payment Model (APM)
- Accountable Care Organization (ACO)
- Global Budget
- Capitation
- Social Determinants of Health (SDOH)
- Health Equity
- Prevention Quality Indicator (PQI)
- HEDIS
- Fee-for-Service
- Value-Based Care
- AHEAD Model
- Hospital-at-Home
- Community Health Worker (CHW)
- Prior Authorization
- Health Information Exchange (HIE)

After creating each definition, search for it at `localhost:3000/academy/glossary` to confirm it appears.

After adding new glossary definitions, run a re-index:

```
curl -X POST http://localhost:8000/api/ingest
```

This ensures the AI Analyst can reference the definitions when answering questions.

6.5 Webinar Creation and Management

Webinars serve two purposes: scheduled live events and permanent "on-demand" recordings.

Creating an upcoming webinar: In Studio → Webinar → + New

```
Title: "Vermont's AHEAD Model: Three Years of Global Budget Implementation"
Description: [AI-generate using Template A context, focused on the webinar topic]
Pillar: Policy
Date: [A future date/time]
Duration: 60 minutes
Registration Link: [Use a placeholder like
https://zoom.us/webinar/register/placeholder for testing]
```

After the event date passes, update the description to indicate it's now on-demand.

Verify at `localhost:3000/academy/webinars`.

Chapter 7 — HTI Dashboard and State Data Analysis

This chapter walks you through every analytical feature in the HTI Dashboard and teaches you enough healthcare economics to evaluate whether the data tells a coherent story.

7.1 Accessing the HTI Dashboard

The HTI Dashboard requires a `professional` or higher role. If testing with a subscriber account:

```
-- Grant professional role in Supabase SQL Editor
INSERT INTO user_roles (user_id, role)
VALUES ('your-user-uuid', 'professional')
ON CONFLICT (user_id, role) DO NOTHING;
```

Navigate to the HTI Dashboard in the main navigation. You should see four tabs: Simulation, Trends, Compare, Clinical.

7.2 Tab 1: Simulation

The Simulation tab lets you calculate an HTI score for a hypothetical institution, region, or state by adjusting sliders for each of the six domains.

Understanding the presets: Three presets load different starting conditions:

- **Hospital** — typical individual facility (composite ≈ 53, "Stable")
- **Region** — multi-county average (composite ≈ 61, "Stable")
- **State** — statewide aggregate (composite ≈ 66, "Improving")

Simulation Exercise 1: Identify the highest-impact domain

Start with the Hospital preset. Note the composite score (≈53).

Now move each domain slider up by 10 points, one at a time, resetting between each:

- Add 10 to Digital Maturity → composite changes by 10 × 0.20 = **+2.0 points**
- Add 10 to Value-Based Care → composite changes by 10 × 0.15 = **+1.5 points**
- Add 10 to Social Determinants → composite changes by 10 × 0.20 = **+2.0 points**
- Add 10 to Clinical Excellence → composite changes by 10 × 0.20 = **+2.0 points**
- Add 10 to Patient Experience → composite changes by 10 × 0.15 = **+1.5 points**
- Add 10 to Workforce Wellness → composite changes by 10 × 0.10 = **+1.0 point**

What this teaches: Digital Maturity, Social Determinants, and Clinical Excellence have the highest leverage on the composite (equal at 20% each). A hospital struggling with equity gaps that invests in digital infrastructure may move the HTI composite more efficiently than investing in workforce wellness — though workforce investment has compounding effects that the linear composite doesn't capture.

Simulation Exercise 2: Modeling Texas moving to Leading

Set the simulation to the State preset. Then manually input Texas's Q1-2025 values:

- Digital: 67, VBC: 52, Equity: 44, Clinical: 68, Experience: 65, Workforce: 54
- Composite: ~58 (Stable)

Now model what it would take for Texas to reach "Improving" (65):

- It needs +7 composite points from 58
- If Texas focused only on Equity (its worst domain at 44), it would need to add 35 points on Equity to gain 7 composite points (35 × 0.20 = 7)
- Alternatively: +7 on Digital (+1.4), +7 on Equity (+1.4), +7 on Clinical (+1.4), +7 on VBC (+1.05), +7 on Experience (+1.05) = 6.3 points total — a distributed approach reaches Improving with more modest gains per domain

What this teaches: Improving from At Risk/Stable requires either concentrated intervention in the highest-weight domains or broad-based improvement across all domains. The simulation makes these tradeoffs concrete.

Evaluating the radar chart: The three-ring radar chart shows:

- **Your institution/state** (green)
- **National Average** (gray dashed)
- **National Excellence Benchmark** (blue dashed)

Any domain where your score falls inside the National Average ring is a gap area. Any domain where you exceed the Excellence Benchmark would be a leading practice.

For a hospital preset, the radar will show:

- VBC and Equity well inside National Average — these are the primary gap domains for most hospitals
- Clinical Excellence close to or above National Average — hospitals tend to invest in clinical quality
- Workforce inside National Average — burnout and turnover are still above sustainable levels nationally

---

7.3 Tab 2: Trends

The Trends tab shows the quarterly HTI trajectory for each of the eight tracked states, with projection for the next four quarters.

Exercise: Vermont Trend Analysis

Select Vermont. Note:

- Composite trajectory: 76 (Q1-2023) → 82 (Q1-2025), velocity +6
- The projection line extends to approximately 83 by Q1-2026
- Select each domain from the dropdown to see the domain-level trend

What to look for in trend analysis:

1. **Acceleration or deceleration:** Is the slope steepening (transformation momentum building) or flattening (approaching ceiling or stalling)?

2. **Domain divergence:** Are some domains improving faster than others? Vermont's VBC domain (already at 88) is likely plateauing while other domains still have room to grow.

3. **Projection credibility:** The projection uses linear extrapolation. For states already in the Leading tier, this tends to *overestimate* future scores because the methodology doesn't account for ceiling effects. Vermont projected at 83 is more believable than Texas projected at 64 — Texas has more room to run before structural barriers slow progress.

Exercise: Compare Texas with National Benchmark

Select Texas. Toggle the projection on.

Observe:

- Texas started at 52 (Stable, lower bound) in Q1-2023
- As of Q1-2025: 58
- Projection: reaching roughly 62–64 by Q1-2026

Ask: Is the projected trajectory sufficient for Texas to reach "Improving" status (65) within 2 years? At +1.5 points per quarter, Texas would reach 65 approximately Q2-2026. This is achievable with current trajectory — but the projection assumes nothing changes. Texas's refusal to expand Medicaid and its high uninsured rate (18.4%) are structural constraints that linear extrapolation ignores.

7.4 Tab 3: Compare

The Compare tab places two states side-by-side with a bar chart comparison across all six domains.

Most useful comparison: Vermont vs. Texas

This is the platform's most powerful teaching comparison because:

- Maximum composite gap: Vermont 82 vs. Texas 58 (24 points)
- The gap is not uniform: VBC gap is 36 points (88 vs. 52); Clinical gap is only 20 points (88 vs. 68)
- The VBC gap reflects Vermont's 20+ years of payment reform policy vs. Texas's fee-for-service-dominant market
- The smaller clinical gap reflects that Texas hospitals, despite the structural environment, still deliver decent individual clinical care — the system-level failures don't fully manifest in facility-level clinical metrics

What to explain to users about this comparison: Texas is not "bad at healthcare." Texas hospitals contain excellent clinicians. The low HTI score reflects *system-level* failures: lack of insurance coverage (18.4% uninsured), lack of Medicaid expansion, minimal VBC infrastructure, and wide equity gaps in outcomes — none of which are problems that individual hospitals or clinicians can solve without policy and economic change.

7.5 Tab 4: Clinical

The Clinical tab shows the 7 supplementary clinical metrics for each state versus the national benchmark.

Critical metrics to examine:

Ohio opioid overdose deaths (42.6 per 100k vs. national 22.4): Ohio's opioid death rate is nearly double the national average. This is one of the most catastrophic public health crises in any state and is *not adequately captured* in Ohio's composite HTI score of 67. This is an important limitation to note when reviewing the clinical data — the composite is a transformation index, not a crisis index.

Texas maternal mortality (34.2 per 100k vs. national 23.5): Texas's maternal mortality rate is 46% above the national average. Among the tracked states, only Florida (28.6) comes close. States that have not expanded Medicaid have significantly higher maternal mortality rates — a direct consequence of access barriers in the perinatal period.

Massachusetts cancer screening rate (82% vs. national 69%): Massachusetts achieves the highest cancer screening rate in the dataset, 13 points above national average. This reflects the combination of near-universal insurance coverage (3.1% uninsured), strong primary care density (19.2 PCPs per 10k), and the Massachusetts All-Payer Claims Database infrastructure that enables population-level screening management.

Evaluating quality of the clinical data: Ask: Do the clinical metric patterns make intuitive sense given each state's policy context? If they do, the data is probably accurate. If a state's clinical metrics are dramatically inconsistent with its HTI scores (e.g., a Leading state with catastrophic maternal mortality), either the data is wrong or the HTI has a blind spot worth investigating.

7.6 The Solvency Simulation

The Solvency Simulation is a separate tool (accessible from the main navigation) that models 12-month cash position for a healthcare institution facing a strategic decision.

Running the simulation:

1. Start with the Status Quo scenario
    - $3.5M monthly burn, no savings, $30M starting cash
    - Expected: Cash drops to $0 by month 9
    - Pass condition: Chart shows cash depleting to zero
2. Switch to Aggressive Consolidation
    - $3.5M burn, $5.5M savings starting month 4, $30M start
    - Expected: Cash ends at ~$37.5M after 12 months
    - Pass condition: Chart shows cash growing after month 3
3. Switch to Regional Partnership
    - $3.5M burn, $3.8M savings starting month 7, $30M start
    - Expected: Cash ends at ~$10.8M (below the $15M viability threshold)
    - The chart should show this as non-viable (viability indicator shows "At Risk")

Educational context for this tool: The solvency simulation is designed to demonstrate a core insight in healthcare financial strategy: *implementation lag is the difference between survival and failure*.

The Aggressive Consolidation scenario generates $2M/month in net savings — but only after a 3-month ramp. The Regional Partnership generates only $0.3M/month in net savings — but doesn't start until month 7. Six months of burn before savings arrive reduces a $30M cushion to $9M, which is too thin to survive unexpected events.

This is why hospital CFOs are deeply risk-averse about transformation timelines. The mathematics of burn rates and implementation lag means that a partnership that would be financially sustainable in month 8 might be financially fatal if pursued two years later when the cash cushion is thinner.

Chapter 8 — Production Deployment

Your application has passed testing. This chapter covers everything needed to deploy publicly. Follow this in order — sequence matters.

8.1 What You're Deploying

Component	Where	Method
Frontend (Next.js)	Vercel	Git-connected auto-deploy
Backend (FastAPI)	Railway	Git-connected auto-deploy
Database	Supabase	Already cloud, no deployment needed
CMS	Sanity	Already cloud, no deployment needed
Payments	Stripe	Switch from test mode to live mode

8.2 Pre-Deployment Checklist

Complete every item before starting deployment.

Content:

- [ ] At least 5 published Policy Analysis articles (one per pillar)
- [ ] At least one complete course (3+ modules) in the Academy
- [ ] At least 10 glossary definitions covering core terms
- [ ] At least 2 Analyst Notes active in The Signal

- [ ] HTI data verified (state scores look accurate)

Technical:

- [ ] All AI chat tests from Chapter 5 pass with scores ≥ 10/15
- [ ] Stripe test checkout completes and role is granted
- [ ] RAG index is current (run `/api/ingest` after last content addition)
- [ ] No console errors in browser DevTools on the homepage, a content page, and the chat page
- [ ] Mobile layout verified on at least two breakpoints (375px and 768px)

Legal/Compliance (minimum):

- [ ] Privacy Policy page exists at `/privacy` or is linked in the footer
- [ ] Terms of Service page exists at `/terms` or is linked in the footer
- [ ] Cookie consent notice (if collecting analytics)

8.3 Railway Setup — Backend Deployment

Railway hosts the Python FastAPI backend.

Step 1: Create Railway Account

Go to `railway.app` → create account → New Project

Step 2: Deploy from GitHub

Connect Railway to your GitHub repository. Select the repository and set the root directory to `/backend`.

Railway will detect the Python project and use nixpacks to build it.

Step 3: Configure Start Command

In Railway: project → Settings → Deploy

Start command:

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

Step 4: Set Environment Variables

In Railway: project → Variables

Add every variable from the backend checklist:

```
OPENAI_API_KEY=sk-...
GROQ_API_KEY=gsk_...
SANITY_PROJECT_ID=fxz10xl7
SANITY_DATASET=production
SANITY_API_TOKEN=sk...
SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_JWT_SECRET=[from Supabase Settings → API → JWT Secret]
SUPABASE_DB_URL=postgresql://postgres.[ref]:[password]@aws-0-us-east-
1.pooler.supabase.com:6543/postgres
FRONTEND_URL=https://your-production-domain.com
INGEST_SECRET=[generate a random 32-character string]
```

Getting `SUPABASE_DB_URL`: Supabase Dashboard → Settings → Database → Connection Pooling → Transaction mode (port 6543). Copy the full connection string.

Getting `SUPABASE_JWT_SECRET`: Supabase Dashboard → Settings → API → JWT Secret. Copy the entire value.

Generating `INGEST_SECRET`:

```
openssl rand -hex 32
```

Step 5: Configure Health Check

In Railway: project → Settings → Healthcheck

```
Path: /health
Timeout: 10 seconds
```

Step 6: Deploy and Verify

Railway will trigger an automatic deploy. Monitor the build logs.

After deployment:

```
curl https://your-railway-app.railway.app/health
```

Expected: `{"status": "ok", "index_ready": false, ...}` initially (index is building)

After 2–3 minutes:

```
curl https://your-railway-app.railway.app/health
```

Expected: `"index_ready": true`

Note your Railway URL — you need it for the Vercel setup.

8.4 Vercel Setup — Frontend Deployment

Vercel hosts the Next.js frontend.

Step 1: Create Vercel Account and Project

Go to `vercel.com` → create account → Add New → Project

Connect to your GitHub repository. Set the root directory to `/frontend`.

Vercel will detect Next.js automatically.

Step 2: Set Environment Variables

In Vercel: project → Settings → Environment Variables

Add all variables from the frontend checklist. For production, use **live** Stripe keys (not test keys):

```
NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01
SANITY_API_TOKEN=[your Sanity token]
NEXT_PUBLIC_SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=[your anon key]
SUPABASE_SERVICE_ROLE_KEY=[your service role key]
STRIPE_SECRET_KEY=sk_live_...    ← LIVE key, not test
STRIPE_WEBHOOK_SECRET=whsec_...  ← Will get this in the next step
STRIPE_PRICE_SUBSCRIBER_MONTHLY=price_live_...
STRIPE_PRICE_SUBSCRIBER_YEARLY=price_live_...
STRIPE_PRICE_STUDENT_MONTHLY=price_live_...
STRIPE_PRICE_STUDENT_YEARLY=price_live_...
STRIPE_PRICE_PROFESSIONAL_MONTHLY=price_live_...
STRIPE_PRICE_PROFESSIONAL_YEARLY=price_live_...
```

```
RESEND_API_KEY=re_...
DIGEST_SECRET=[generate a random 32-character string]
PYTHON_BACKEND_URL=https://your-railway-app.railway.app
NEXT_PUBLIC_APP_URL=https://your-production-domain.com
```

Stripe live keys vs. test keys: In Stripe: toggle the mode switch to "Live" at the top-left of the dashboard. Create new products and prices for live mode (or verify test products exist in live mode — they don't; you must recreate them). Get live secret key: Developers → API keys → Secret key (starts with `sk_live_`).

Step 3: Deploy

Vercel will auto-deploy when you push to `main`. You can also trigger a manual deploy from the Vercel dashboard.

After deploy, note your Vercel URL (e.g., `your-project.vercel.app`).

Step 4: Custom Domain

In Vercel: project → Settings → Domains

Add your production domain (e.g., `healthtransformationreport.com`).

Follow Vercel's DNS instructions to add CNAME or A records with your domain registrar.

After DNS propagates (5–30 minutes), your site is live at your domain.

8.5 Stripe Production Webhook Setup

With your production domain live, register the Stripe webhook:

1. Stripe Dashboard (in Live mode) → Developers → Webhooks → Add endpoint
2. URL: `https://your-domain.com/api/stripe/webhook`
3. Events to listen for:
    - `checkout.session.completed`
    - `customer.subscription.updated`
    - `customer.subscription.deleted`
    - `invoice.payment_failed`
4. Click "Add endpoint"
5. Copy the signing secret (starts with `whsec_`)
6. Update `STRIPE_WEBHOOK_SECRET` in Vercel environment variables
7. Trigger a Vercel redeploy to pick up the new env var

Test the production webhook: In Stripe dashboard: Webhooks → your endpoint → Send test webhook → `checkout.session.completed`

Verify it shows "200" in the webhook delivery log.

8.6 Railway CORS Update

Update the `FRONTEND_URL` variable in Railway to your production domain:

```
FRONTEND_URL=https://your-domain.com
```

Redeploy Railway after updating. Without this, the backend will reject API requests from the frontend.

8.7 Trigger Production RAG Index

With everything deployed:

```
curl -X POST https://your-railway-app.railway.app/api/ingest \
  -H "Authorization: Bearer your-ingest-secret"
```

Monitor Railway logs for the index build completion.

This is critical — the production backend starts fresh and needs to build its knowledge base.

8.8 Post-Deployment Smoke Tests

Run these tests against your production URL before announcing the launch:

Test 1: Homepage loads

- Go to `https://your-domain.com`
- Header, ticker, sidebars visible
- No console errors (open browser DevTools → Console)

Test 2: Content is live

- Go to a published Policy Analysis article
- Go to an Academy Module
- Go to `/academy/glossary` and search for a term

Test 3: Signup and login

- Create a new account
- Verify email confirmation arrives (if email confirmation is enabled in Supabase)
- Log in successfully

Test 4: Stripe checkout (LIVE MODE — small charge)

- While logged in, go to `/pricing`
- Attempt checkout for the Subscriber plan monthly
- Use a real card (this will charge $29)
- Verify role is granted in Supabase
- Verify the Stripe webhook shows 200 in the live webhook log
- Immediately cancel the subscription in Stripe to get a refund

Test 5: AI Analyst access

- While logged in as a subscriber, go to `/chat`
- Ask: "What is the AHEAD Model in Vermont?"
- Verify a grounded, coherent response

Test 6: Backend health

```
curl https://your-railway-app.railway.app/health
```

Confirm `"index_ready": true` and `"auth_enabled": true`

Note: In production with `SUPABASE_JWT_SECRET` set, `auth_enabled` will be `true`. This means only users with subscriber+ roles can access the AI.

8.9 Production Monitoring Setup

These are the monitoring tasks that should become routine:

Daily (first week post-launch):

- Check Railway deployment logs for errors
- Check Stripe → Webhooks → recent deliveries (all should show 200)
- Monitor Supabase for unusual query load

Weekly:

- Run the weekly digest email: `POST /api/digest` with `DIGEST_SECRET`

- After publishing new content: run `POST /api/ingest` with `INGEST_SECRET`

When something breaks: Refer to the TROUBLESHOOTING.md guide, which covers the most common failure patterns with diagnostic steps.

8.10 Pre-Launch Content Quality Final Review

Before going public, do one complete content pass:

1. **Read every published Policy Analysis.** Does each one have: a specific policy reference, at least two data points, a clear "so what" for healthcare executives?
2. **Take the first module of every published course.** Could someone with zero background understand it? Does it define every term it uses?
3. **Ask the AI 10 questions** from the Chapter 5 battery. Are all responses at score 10+ on the rubric?
4. **Check every HTI domain score for every state.** Do the scores make intuitive sense? (Massachusetts leading, Texas trailing, Vermont high on VBC — these are structurally correct) Does any state have a score that seems inconsistent with what you know about that state's healthcare system?
5. **Test the glossary.** Search for 10 terms. Do they all appear? Are the definitions clear, accurate, and appropriately concise?
6. **Test navigation.** Click through every item in the main navigation header. Do all five pillar dropdowns lead to populated content pages?

8.11 Go/No-Go Decision

Go if:

- ☐ All smoke tests pass
- ☐ Stripe checkout (live) completes and role is granted
- ☐ AI Analyst answers 8/10 test questions at score ≥10
- ☐ At least 20 pieces of published content across at least 3 pillars
- ☐ No critical console errors on any core page
- ☐ Privacy Policy and Terms of Service are accessible

No-go if:

- ☐ Backend index_ready is false
- ☐ Stripe webhook is not returning 200 in live mode
- ☐ AI Analyst returns "Cannot reach backend" or access denied errors
- ☐ Content library is empty or has fewer than 10 published pieces
- ☐ Any test user cannot complete the signup → subscribe → AI access flow

---

Appendix A — Quick Reference: Key URLs

Resource	Local	Production
Site	`localhost:3000`	`https://your-domain.com`
Studio	`localhost:3000/studio`	`https://your-domain.com/studio`
AI Chat	`localhost:3000/chat`	`https://your-domain.com/chat`
Pricing	`localhost:3000/pricing`	`https://your-domain.com/pricing`
Academy	`localhost:3000/academy`	`https://your-domain.com/academy`

Resource	Local	Production
Glossary	`localhost:3000/academy/glossary`	`https://your-domain.com/academy/glossary`
Backend Health	`localhost:8000/health`	`https://your-railway.railway.app/health`
Supabase	`supabase.com/dashboard/project/clryhwqaqhvdikgesjbc`	Same
Sanity	`sanity.io/manage`	Same
Stripe	`dashboard.stripe.com`	Same (toggle to Live)
Railway	`railway.app`	Same
Vercel	`vercel.com`	Same

Appendix B — Test Card Numbers (Stripe Test Mode Only)

Card Number	Behavior
`4242 4242 4242 4242`	Always succeeds
`4000 0000 0000 0002`	Always declines
`4000 0000 0000 0341`	Succeeds but next payment fails
`4000 0025 0000 3155`	Requires 3D Secure authentication

Appendix C — Rebuilding After Content Updates

Whenever you add significant new content to Sanity:

```
# Production
curl -X POST https://your-railway-app.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"

# Local development
curl -X POST http://localhost:8000/api/ingest
```

The rebuild takes 1–3 minutes. During this time the existing index remains active — users can still use the AI Analyst. The new index replaces the old one atomically when the build completes.

Appendix D — The Weekly Digest Email

The digest sends the 5 most recently published Policy Analysis articles to all active subscribers with digest enabled.

```
# Production
curl -X POST https://your-domain.com/api/digest \
  -H "Authorization: Bearer $DIGEST_SECRET"

# Expected response
{"message": "Digest sent", "sent": 142, "errors": 0, "total": 142}
```

Automate with GitHub Actions (see OPERATIONS_GUIDE.md for the workflow YAML).

Health Transformation Review — Testing and Deployment Guide | Q1 2026 This guide should be reviewed and updated after each significant platform change.